Safeguarding Graph Neural Networks against Topology Inference Attacks

Jie Fu
Stevens Institute of Technology
Hoboken, NJ, USA
jfu13@stevens.edu

Yuan Hong
University of Connecticut
Storrs, Connecticut, USA
yuan.hong@uconn.edu

Zhili Chen
East China Normal University
Shanghai, China
zhlchen@sei.ecnu.edu.cn

Wendy Hui Wang
Stevens Institute of Technology
Hoboken, NJ, USA
hwang4@stevens.edu

Abstract

Graph Neural Networks (GNNs) have emerged as powerful models for learning from graph-structured data. However, their widespread adoption has raised serious privacy concerns. While prior research has primarily focused on edge-level privacy, a critical yet underexplored threat lies in *topology privacy* — the confidentiality of the graph's overall structure. In this work, we present a comprehensive study on topology privacy risks in GNNs, revealing their vulnerability to graph-level inference attacks. To this end, we propose a suite of *Topology Inference Attacks* (TIAs) that can reconstruct the structure of a target training graph using only black-box access to a GNN model. Our findings show that GNNs are highly susceptible to these attacks, and that existing edge-level differential privacy mechanisms are insufficient as they either fail to mitigate the risk or severely compromise model accuracy. To address this challenge, we introduce *Private Graph Reconstruction* (PGR), a novel defense framework designed to protect topology privacy while maintaining model accuracy. PGR is formulated as a bi-level optimization problem, where a synthetic training graph is iteratively generated using meta-gradients, and the GNN model is concurrently updated based on the evolving graph. Extensive experiments demonstrate that PGR significantly reduces topology leakage with minimal impact on model accuracy. Our code is anonymously available at https://github.com/JeffffffFu/PGR.

CCS Concepts

• **Security and privacy**; • **Computing methodologies** → **Machine learning**;

Keywords

Topology privacy, Graph neural networks, privacy inference attacks, privacy and security in machine learning

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CCS '25, Taipei, Taiwan
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1525-9/2025/10
https://doi.org/10.1145/3719027.3765173

ACM Reference Format:
Jie Fu, Yuan Hong, Zhili Chen, and Wendy Hui Wang. 2025. Safeguarding Graph Neural Networks against Topology Inference Attacks. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25), October 13–17, 2025, Taipei, Taiwan.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3719027.3765173

1 Introduction

Graph data is pervasive in real-world applications, ranging from social networks [9] to bioinformatics [30] and recommendation systems [15]. Graph Neural Networks (GNNs) [10, 18] have recently demonstrated remarkable effectiveness in learning from such data.

As GNNs are increasingly applied in various domains and applications, concerns have arisen regarding the data privacy risks associated with GNNs. Among these concerns, edge-level privacy has gained significant attention. Recent studies have demonstrated that the adversary can infer the presence of specific edges in a graph by simply querying a GNN trained on the graph and analyzing its outputs [13, 14, 20, 34, 37, 42]. Several privacy-preserving mechanisms (e.g., [19, 28, 31, 37, 39]) have been designed to provide edge-level privacy protection with the differential privacy (DP) guarantee [5, 6].

While extensive research has primarily focused on edge-level privacy in GNNs, *topology-level privacy* has been largely overlooked. Topology privacy refers to protecting the overall structure of the graph, including both the nodes and the connections between them in the training graph of the target model. Since the graph's structure encodes critical information about the relationships and dependencies among its nodes, revealing the graph's topology goes far beyond simply exposing a few edges. Inferring the graph topology can uncover hidden patterns and relationships within the network, which may be far more sensitive than knowledge of individual connections. For instance, in a social network graph, inferring the topology could enable an adversary to identify vulnerable or isolated communities and strategically target key individuals, such as influential users or celebrity figures, within those communities. Given the significance of topology privacy, it is crucial to investigate the vulnerabilities in GNNs regarding topology leakage and explore potential countermeasures.

In this paper, we take the first step toward studying the topology privacy of GNNs. We conduct a comprehensive investigation into the topology privacy risks in GNNs, uncovering their vulnerability to graph-level inference attacks. Furthermore, we propose effective

countermeasures to protect the topology privacy of GNNs. We make the following contributions.

Attack design and evaluation. We propose three Topology Inference Attacks (TIAs) - *metric-based TIA (M-TIA)*, *classifier-based TIA (C-TIA)*, and *influence-based TIA (I-TIA)* - to reconstruct the overall structure of a private graph through black-box access to GNNs. To quantify the extent of topology privacy leakage (TPL), we define a metric based on the Jaccard similarity between the private graph and the graph inferred by a TIA. Our empirical results demonstrate the effectiveness of these attacks, with I-TIA achieving 100% TPL, indicating complete leakage of the original graph's topology. We further evaluate six state-of-the-art edge-level differential privacy (edge-DP) methods [19, 28, 31, 37, 39] against the proposed TIAs. The results reveal that none of these methods provide sufficient defense: they either fail to substantially reduce TPL or severely degrade model accuracy, leaving GNNs vulnerable to topology privacy attacks.

Design of PGR. We propose a novel algorithm, *Private Graph Reconstruction (PGR)*, to mitigate topology privacy leakage in GNNs while preserving model performance. PGR is formulated as a bi-level optimization problem that aims to generate a synthesized graph $\hat{G}$ which contains no edges from the private training graph G, yet enables any GNN trained on $\hat{G}$ to achieve performance comparable to one trained on G. To solve this optimization problem, PGR iteratively constructs $\hat{G}$ using meta-gradients while simultaneously updating the GNN on the synthesized graph. Furthermore, we introduce *DP-PGR*, an extension of PGR with edge-level differential privacy guarantees, thereby providing protection at both the topology and edge levels.

Evaluation. We conduct extensive experiments on four real-world graph datasets and three widely-used GNN models to evaluate the performance of PGR. The results demonstrate that PGR is highly effective in protecting topology privacy, significantly reducing topology privacy leakage. Moreover, PGR consistently outperforms six state-of-the-art edge-DP approaches [19, 28, 31, 37, 39] in balancing model accuracy and topology privacy. Notably, PGR incurs only a negligible loss in model accuracy, with a maximum decrease of just 0.1%. Furthermore, we evaluate the robustness of PGR against adversaries who possess prior knowledge of its design and parameters. Our findings show only a marginal decline (at most 0.4%) in the effectiveness of PGR, further demonstrating its resilience in protecting topology privacy.

In summary, we make the following contributions.

- We demonstrate that even simple adaptations of existing edge-level inference attacks can lead to significant topology leakage, uncovering a critical and previously underexplored privacy vulnerability in GNNs.
- We empirically show that six state-of-the-art edge-level DP methods are insufficient for protecting topology privacy.
- We introduce PGR, the first defense mechanism specifically designed to mitigate topology privacy leakage in GNNs while maintaining model accuracy.
- We perform extensive empirical evaluations of PGR, demonstrating its ability to effectively balance topology privacy and model utility, while also maintaining robustness against adversaries with advanced knowledge.

2 Related Work

2.1 Privacy Inference Attacks against GNNs

Recent studies have revealed that GNNs are vulnerable to the membership inference attacks (MIAs) [4, 22, 36]. Most existing MIAs against GNNs focus on edge inference. Specifically, a *link membership inference attack* (LMIA) seeks to infer the presence of specific edges in the training graph of a GNN. LMIAs can be broadly classified into two categories: (i) *similarity-based LMIAs* [14, 34, 35], which leverage the intuition that nodes with higher similarity are more likely to be connected, and (ii) *influence-based LMIAs* [13, 20, 37], which use the influence between node pairs for inference. Influence is estimated by applying perturbations to either the node features [13, 37] or the node's neighborhood [20]. A higher influence between two nodes indicates a greater likelihood that an edge exists between them. We will demonstrate that even a simple adaptation of LMIAs can compromise topology privacy (Section 4).

More recently, a new class of attacks, referred to as *graph reconstruction attacks*, has emerged. These attacks aim to reconstruct the training graph [4, 36, 41] through access to GNN models. Wang et al. [36] introduced a black-box attack, SMIA, designed to recover specific graph structures (e.g., k-hop paths or cliques) from the training graph, differing from TIA in its attack goal. Zhang et al. [41] proposed a white-box attack that reconstructs a graph with similar graph statistics (e.g., degree distribution, local clustering coefficient) to the target graph by leveraging its embedding. Duddu et al. [4] also developed a white-box attack that reconstructs the graph using an encoder-decoder model on graph embeddings. However, TIA differs fundamentally from these attacks as it only requires access to the posterior probabilities output by the model, rather than node embeddings [4] or graph embeddings [41].

2.2 Differentially Private GNNs

Differential privacy (DP) [5] is a robust mathematical framework that formally defines the privacy guarantees an algorithm can provide. DP can be applied to graph data in two primary ways [28]: *edge-level DP* and *node-level DP*. In edge-level DP, two graphs are considered neighbors if they differ by a single edge, while node-level DP considers two graphs neighbors if they differ by the edges connected to a single node. Achieving node-level DP can be particularly challenging, as modifying one node may remove up to $K-1$ edges in the worst case, where K is the number of edges in the graph. Therefore, this paper focuses solely on edge-level DP.

Recently, there have been efforts to provide formal edge-level DP privacy guarantees within various GNN learning settings. Wu et al. [37] introduced methods such as *EdgeRand* and *LapEdge*, which perturb the input graph using randomized response or the Laplace mechanism, respectively. The perturbed adjacency matrix is then used for GNN training. Tang et al. [31] proposed *Eclipse*, a method that transforms the adjacency matrix into a low-rank form using singular value decomposition (SVD) before adding Laplace noise to the singular values. Yuan et al. [39] developed *PrivGraph*, an edge-DP approach that partitions nodes into communities and adds noise to both intra-community and inter-community edges. The graph is then synthesized using the CL model [1]. Sajadmanesh et al. [28] designed *GAP*, which achieves edge-DP by adding calibrated Gaussian noise to the output of the GNN's aggregation function.

Symbol	Meaning
$G/\hat{G}$	Original graph/synthetic graph
$f/\hat{f}$	GNN trained on $G/\hat{G}$
G_T/G_A	Target graph/Attack graph
$E/E_T/E_A$	Edge set of $G/G_T/G_A$
$K/\hat{K}$	# of edges of $G/\hat{G}$
K_T/K_A	# of edges in G_T/adversary knowledge

Table 1: Common notations

Kolluri et al. [19] introduced *LPGNet*, which perturbs cluster degree vectors with Laplace noise during training, using these noisy vectors as input features for model training. However, as we will show in Section 4, these edge-DP methods fail to provide adequate protection for topology-level privacy.

2.3 Graph-level Privacy Protection

Most existing graph-level privacy protection methods [12, 26, 40] focus on the Federated Learning (FL) setting, aiming to protect the privacy of local graphs on the client side. Specifically, Han et al. [12] address privacy in recommendation systems, expanding local graphs leveraging Software Guard Extensions (SGX) and Local Differential Privacy (LDP). Qiu et al. [26] propose *DP-FedRec*, a federated GNN that employs Private Set Intersection (PSI) to extend local graphs for each client, tackling the non-IID challenge in the FL setting. Zhang et al. [40] take a subgraph-level approach, utilizing the *information bottleneck principle* to extract task-relevant subgraphs from local client graphs for training and local model updates. While these methods address challenges like non-IID distributions and safe gradient sharing within the FL framework, their goals and strategies differ fundamentally from ours.

3 Problem Formulation

Given an undirected graph $G(V, E)$, where V and E denote the nodes and edges of G, respectively, a GNN model aims to learn a low-dimensional embedding for each node by leveraging both the node features and the overall topology of the graph. The learned node embeddings can then aid downstream graph learning tasks, such as node classification, link prediction, and clustering. In this paper, we consider node classification as the learning task. Specifically, for any given node $v \in G$, the GNN model outputs a *posterior probability vector* for v, in which the i-th entry indicates the probability that v is associated with the i-th label. Specifically, we consider node classification in the transductive setting, given its wide use in practice. Table 1 lists the common notations used in the paper.

3.1 Threat Model

An attacker may seek to infer topological information about the training graph of a GNN model. This information could pertain to either the entire graph or a specific subgraph. For example, consider an attacker planning a cyberattack on a computer network. The attacker might target the entire network, attempting to reconstruct the topology of the full network graph, or focus on a particular subnet, aiming to infer the structure of the corresponding subgraph.

Therefore, given a graph $G(V, E)$ and a GNN model f trained on G, the adversary's goal is to infer the topology of a *target graph* $G_T(V_T, E_T) \subseteq G$ where $V_T \subseteq V, E_T \subseteq E$. We assume the adversary has the following knowledge:

- **Black-box model access.** Following the prior works [14], we assume the adversary has black-box access to f. Specifically, the adversary can access $f(v)$ for any query node v, where $f(v)$ outputs a posterior probability vector. Several machine-learning-as-a-service providers, such as Google Cloud's Vertex AI [8], Facebook's ParlAI [23], and IBM's InfoSphere Virtual Data Pipeline [16], offer such black-box access. Consistent with existing LMIAs [14, 37], we also assume the adversary has an unlimited query budget.
- **Auxiliary graph information.** We consider an adversary who has knowledge of the nodes and their features in the training graph - and consequently in the target graph G_T - as assumed in prior works [14, 37]. In addition, the adversary possesses the auxiliary information about the number of edges in G_T. This estimate is presumed to be close to the true value, i.e., $K_A \approx K_T$, where K_T represents the actual number of edges in G_T. Such information is often readily available from external sources and may even be publicly accessible, even when the entire training graph is the attack target. For instance, data agencies like Statista [3] frequently publish statistics on the number of nodes and edges for major social networks, such as Facebook and Instagram.

Given these knowledge, the attacker's goal is to infer a graph $G_A(V_A, E_A)$ where $V_A = V_T$ and $|E_A| = K_A$. Figure 1 illustrates an example of the attack at a high level. Compared with existing LMIAs [14, 37], our adversary has additional knowledge about the number of edges in the training graph. We will show that such a simple piece of adversary knowledge will bring the privacy risks to the graph topology, revealing new privacy vulnerabilities in GNNs.

As the graph structure carries a wealth of information about the relationships and dependencies between its nodes, revealing graph topology goes beyond simply exposing a few edges. The inference of graph topology can reveal hidden structures and relationships in the network, which can be far more sensitive than just knowing a few individual connections. For example, as the topology of a graph often reveals clusters or communities of closely connected nodes, it allows the adversary to identify the existence of these communities, leading to potential target attacks against specific user groups. Furthermore, deriving a node's degree or its centrality in the network from the topology can help the attacker to target key players or potential weak points in the network.

3.2 Measurement of Topology Privacy Leakage

Measuring topology privacy leakage is crucial for understanding how much sensitive information about a network's structure might be exposed. By quantifying this leakage, we can assess the risk of revealing key characteristics such as node relationships and connectivity patterns within the network. In this paper, we consider similarity measures to help evaluate the extent of the training graph's topology being inferred by the attacks. [1]

[1]We also examined an alternative TPL metric and found its results to be consistent with those obtained using the Jaccard similarity metric. Further details are provided in Section 7.

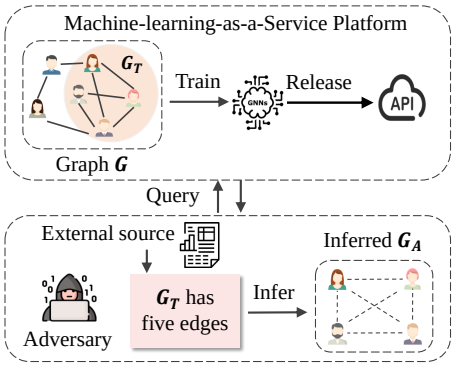


Figure 1: An example of the topology inference attack. The target graph G_T, which is a subgraph of G, is shaded.

Specifically, given a graph $G(V, E)$ and a GNN model f trained on G, let $G_T(V_T, E_T)$ be the private graph, where $V_T \subseteq V, E_T \subseteq E$. Let G_A be the *attack graph* inferred by the adversary through black-box access to f, and E_A be the edges of G_A. We quantify the *topology privacy leakage* of f as the Jaccard similarity between E_T and E_A:

$$TPL = \frac{|E_T \cap E_A|}{|E_T \cup E_A|}. \tag{1}$$

The Jaccard similarity is directly related to True-Positive (TP), False-Positive (FP), and False-Negative (FN) edges, where TP edges are those that are present in both graphs, FP edges are those that appear in E_A but not in E_T, and FN edges are those are missing in E_A but are present in E_T. Specifically, the numerator $|E_T \cap E_A|$ corresponds to the TP edges, while the denominator corresponds to the total number of unique edges that appear in either E_T or E_A, which is calculated as $|TP| + |FP| + |FN|$. Thus, the Jaccard similarity can be written in terms of TP and FP as: $TPL = \frac{|TP|}{|TP|+|FP|+|FN|}$. A higher Jaccard similarity indicates that the graphs are more similar, meaning there are more true-positive edges relative to false positives and false negatives. Conversely, a lower Jaccard similarity means the predicted graph has many false positives or false negatives, indicating less accuracy in representing the true graph's topology. TPL reaches the maximum (i.e., $TPL = 1$) when $E_T = E_A$, and the minimum (i.e., $TPL = 0$) when $E_T \cap E_A = \emptyset$.

4 Topology Inference Attack

Given the adversary knowledge, the attacker performs a *topology inference attack* (TIA) to construct a graph whose topology is consistent with the known number of edges. Formally, given a training graph $G(V, E)$, let f be the GNN model trained on G, and $G_T(V_T, E_T) \subseteq G$ be the target private graph. The adversary aims to construct an attack graph $G_A(V_T, E_A)$ where $|E_A| = K_A$, through black-box access to f.

Intuitively, the adversary can construct all possible graphs whose number of edges equals to K_A, and randomly pick one from these graphs as G_A. However, such naive method is prohibitively costly due to its large search space.[2] Furthermore, as it neglects the output of the GNN model, its accuracy is poor (as shown in Appendix A). Therefore, we aim to design more effective TIAs through utilizing the black-box access of f.

Intuitively, the graph topology can be inferred by reconstructing the individual edges. Building upon this intuition, we extend existing LMIAs [14, 20, 37] to TIAs, broadening the inference from a single edge to a set of edges. Our goal is to show that topology inference does not require the design of entirely new attack strategies - rather, even straightforward adaptations of existing LMIAs can pose significant threats to topology privacy.

4.1 Attack Details

Based on the underlying reasoning of the attacks, we classify our TIAs into two types: *similarity-based* and *influence-based*. Next, we present the details of each type of attacks.

Similarity-based Topology Inference Attack. In this attack, the adversary constructs edges in G_A based on the similarity between the posterior probability of nodes. Specifically, we design two TIAs that exploit this similarity to infer the graph structure:

- **Metric-based TIA (M-TIA)**: The adversary computes the similarity between the posterior probability vectors of each node pair (u_i, v_j) in graph G using a distance-based metric.[3] The top K_A node pairs with the highest similarity scores are then selected, and edges are inserted between them in G_A.
- **Classifier-based TIA (C-TIA)**: The adversary leverages an existing LMIA, such as [14, 35], to construct G_A. Specifically, for each node pair (v_i, v_j), the adversary queries the LMIA classifier to obtain both a membership prediction label and a corresponding confidence score. Among the node pairs predicted as members (i.e., links), those with the top K_A highest confidence scores are selected and inserted as edges in G_A.

Influence-based Topology Inference Attack (I-TIA). We adapt the influence-based LMIAs [20, 37] to the topology inference setting. Specifically, for each node pair (v_i, v_j), we compute the influence of node v_i on node v_j. In this work, we adopt the pairwise influence score function proposed in [20]. The K_A node pairs with the highest influence scores are then selected and added as edges in G_A.

While our TIAs are inspired by existing edge-level inference attacks, they represent a significant advancement by moving beyond isolated link prediction to reconstruct broader topological structures. As we will demonstrate in Section 4.2, even such simple adaptations of edge-level attacks can lead to significant topology leakage, highlighting a critical and previously underexplored privacy vulnerability in GNNs.

4.2 Insufficiency of Edge-DP as Defense

There have been active attempts to provide edge-level differential privacy (edge-DP) in GNNs. This raises the following concern: *While these approaches can provide rigorous privacy protection at the edge level, are they able to ensure privacy at the topology level? Specifically, what will be the privacy leakage of these approaches against TIAs?* We will answer this question through empirical evaluations.

[2] Given N nodes, there are $\binom{N(N-1)/2}{K}$ possible graphs that have the same N nodes and K edges.

[3] We consider three distance functions: Cosine distance, Chebyshev distance, and Euclidean distance. In our experiments, we report the highest attack accuracy achieved across these three metrics.

Table 2: Topology privacy leakage (%) and node classification accuracy (Utility) of GCN model with and without edge-DP protection. The privacy budget of all edge-DP approaches is set as $\epsilon = 7$.

Approach	Cora				LastFM				Emory			
	M-TIA	C-TIA	I-TIA	Utility	M-TIA	C-TIA	I-TIA	Utility	M-TIA	C-TIA	I-TIA	Utility
No Edge-DP	28.10	25.53	100.00	83.38	25.04	25.11	100.00	81.75	17.55	16.54	100.00	90.68
Eclipse [31]	18.58	16.46	29.50	72.45	16.48	5.93	20.94	73.87	20.07	15.91	27.36	83.78
LPGNet [19]	24.67	24.23	17.12	70.13	19.67	22.74	13.76	56.82	18.96	19.38	17.16	71.38
PrivGraph [39]	2.94	1.89	13.38	43.88	19.21	18.34	13.87	46.89	17.06	11.56	14.74	50.43
GAP [28]	4.63	3.86	1.47	59.45	14.63	16.53	4.25	67.55	13.89	11.54	12.06	67.97
LapEdge [37]	26.11	25.02	72.24	61.63	25.01	25.01	58.12	54.89	18.29	15.40	95.30	87.10
EdgeRand [37]	25.52	25.08	96.22	74.04	19.63	18.41	98.66	69.75	17.52	13.89	98.98	89.05

Experimental setup. For this part of evaluation, we set up a two-layer Graph Convolutional Networks (GCN) [32] with a 32-neuron hidden layer as the target model. We utilize the ReLU activation function and Softmax function for the output. We consider six state-of-the-art edge-DP approaches: *Eclipse* [31], *LPGNet* [19], *PrivGraph* [39], *GAP* [28], *LapEdge* [37], and *EdgeRand* [37].

We conducted the experiments on four real-world graph datasets. [4] For each dataset, we sample five subgraphs. [5] Each subgraph contains 100 nodes (~5%-10% of the original graph). We consider these sampled subgraphs as the target private graphs.

Regarding the implementation of TIAs, we utilized *StealLink* [14] for both M-TIA and C-TIA, and *Link-Infiltrator* [20] for I-TIA. We do not use the same LMIA across three types of TIAs as StealLink is not suitable for I-TIA as it does not rely on the node influence for privacy inference, while Link-Infiltrator cannot be used for both M-TIA and C-TIA as it does not utilize node similarities. In our evaluation, we assume that the adversary has knowledge of the exact number of edges in the target private graph (i.e., $K_A = |E_T|$). This assumption allows for the strongest possible attack, which will serve as the primary basis for designing our defense strategy.

To assess the privacy risk of GNNs against TIAs, we measure the topology privacy leakage (Eqn. (1)) of the target GNN model with and without edge-DP protection. Furthermore, we measure *model utility* as the *accuracy of node classification*, i.e., the percentage of nodes whose labels are correctly predicted.

Results. Table 2 presents the results of both topology privacy leakage (TPL) and model utility across three types of TIAs, with the privacy budget ϵ set to 7.[6] The TPL result is reported as the average TPL across the five subgraphs, whereas model utility is evaluated on the entire testing graph.

First, we observe that in the absence of edge-DP protection, the GCN model is consistently vulnerable to all three TIAs, exhibiting substantial topology privacy leakage (TPL) across all settings. Among the three, the model is particularly susceptible to I-TIA, with TPL reaching as high as 100%. This suggests that the node influence exploited by I-TIA more effectively captures structural information than M-TIA and C-TIA, likely due to its alignment with the message-passing mechanism inherent to GNNs.

Second, although all six edge-DP approaches demonstrate some level of protection against TIAs, their effectiveness varies significantly. In particular, LapEdge and EdgeRand provide the weakest protection, whereas GAP consistently offers the strongest defense, in most settings. The limited effectiveness of LapEdge and EdgeRand stems from their inability to alter the ranking of nodes' similarity and influence - rankings that are computed from posterior probabilities. Although these probabilities are perturbed, the relative order often remains unchanged, resulting in reconstructed graphs that closely resemble the original despite the added noise. In contrast, GAP is capable of perturbing both the nodes' posterior probabilities and their induced rankings, thereby demonstrating strong effectiveness against TIA.

However, although certain edge-DP approaches - such as GAP and PrivGraph - are effective in significantly reducing TPL, they incur substantial degradation in model utility. For example, on the Cora dataset, GAP reduces the model utility from 83.38% to 59.43%, which is close to random guessing in a binary classification task.

In summary, none of the six edge-DP methods offers a satisfactory defense against TIAs. They either fail to significantly reduce the TPL or cause a substantial drop in model accuracy, making them ineffective as mitigation solutions.

5 PGR: Our Countermeasure of TIA

Intuitively, the minimal topology privacy leakage (Eqn. (1)) occurs when there is no overlap between the edges of G_T and G_A. However, the defender typically lacks prior knowledge of the target private graph G_T. Therefore, the defender must consider all possible subgraphs as potential attack targets. The most effective strategy to protect against all such possibilities is to construct a new graph $\hat{G}$ whose edge set is entirely disjoint from that of the training graph G. A new GNN model, $\hat{f}$, is then trained on $\hat{G}$. Since $\hat{G}$ contains no edges from G, the model is expected to reveal minimal structural information about both G and thus any potential target graph.

Following this intuition, we design PGR, a new privacy protection method to mitigate the TPL of GNN models. PGR aims to serve a dual goal:

- **Goal 1 (Privacy)**: Given a graph $G(V, E)$, constructs a graph $\hat{G}(V, \hat{E})$ such that $\hat{E} \cap E = \emptyset$.
- **Goal 2 (Model accuracy)**: The performance of the model $\hat{f}$ trained on $\hat{G}$ is close to that of f trained on G.

Figure 2 illustrates the framework of PGR. Next, we discuss the details of PGR.

[4]The descriptions and statistics of these datasets can be found in Table 3.

[5]We randomly select a node from the graph as the starting point, and utilize the Breadth First Search (BFS) strategy to obtain edges in the sampled graph.

[6]We chose $\epsilon = 7$ as it best strikes the balance between model accuracy and edge-level privacy. Additional results on the privacy-accuracy trade-off can be found in Appendix B.

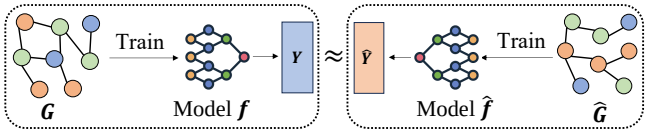


Figure 2: Illustration of PGR.

5.1 Bi-level Optimization

First, we formalize the optimization problem of PGR.

Admissible space of graph construction. Intuitively, the topology privacy leakage is minimized if G and $\hat{G}$ do not share any edge (i.e., $E \cap \hat{E} = \emptyset$). Therefore, we define the *admissible space* (denoted as $\Phi(G)$) of $\hat{G}$ w.r.t. a given graph G is thus defined as

$$\Phi(G) = \{\hat{G} \mid E \cap \hat{E} = \emptyset\},$$

where E and $\hat{E}$ are the edge set of G and $\hat{G}$, respectively. $\hat{G}$ have the same sets of labeled nodes as f. These nodes in $\hat{G}$ are associated with the same labels as their corresponding counterparts in G. The node features of $\hat{G}$ remain the same as G.

Objective function. Recall that one of the goals of PGR is to ensure the performance of the model $\hat{f}$ trained on $\hat{G}$ is close to that of the model f trained on G. We formulate this as a bi-level optimization problem, with $\hat{G}$ and $\hat{\theta}$ as the outer-level and inner-level variables:

$$\begin{aligned} \hat{G}^* &= \overbrace{\arg\min_{\hat{G} \in \Phi(G)} \mathcal{L}_{gen}\left(\hat{f}(\hat{G}; \theta^*), Y_P\right)}^{\text{outer-level objective}} \\ \text{s.t.} \quad \theta^* &= \underbrace{\arg\min_{\theta} \mathcal{L}_{train}\left(\hat{f}(\hat{G}; \theta), Y_L\right)}_{\text{Inner-level objective}} \end{aligned} \tag{2}$$

where $\Phi(G)$ represents the admissible space of $\hat{G}$, Y_L denotes the labels of the labeled nodes in G, and Y_P denotes the predicted labels of the unlabeled nodes in G predicted by f. $\mathcal{L}_{gen}$ and $\mathcal{L}_{train}$ represent the loss function of node classification evaluated on Y_P and Y_L, respectively. [7] In other word, the inner-level objective is to train a model $\hat{f}$ on the graph $\hat{G}$, while the outer-level objective is to generate the graph $\hat{G}$ such that the model $\hat{f}$ trained on $\hat{G}$ approximates the performance of the model f on G.

5.2 Meta-Gradient

Exactly solving Eqn. (2) requires computing the gradient of the outer objective with respect to $\hat{G}$, which in turn involves fully solving the inner optimization problem to obtain the model parameters θ (i.e., until convergence). This process is computationally intensive. To address this, we propose an approximate solution based on the meta-gradient technique. Specifically, we treat the number of edges $\hat{K}$ in the reconstructed graph $\hat{G}$ as a hyperparameter, and use the meta-gradient approach [33] to efficiently search for a graph with $\hat{K}$ edges that enables the GNN to achieve prediction accuracy comparable to that trained on the original graph.

Intuitively, meta-gradients approximate the optimal solution to a bi-level optimization problem by using truncated optimization — that is, updating the inner model parameters for only a few steps

[7] We use cross-entropy for both $\mathcal{L}_{train}$ and $\mathcal{L}_{gen}$.

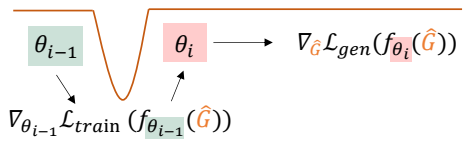


Figure 3: Computation of the meta-gradient $\nabla^{\text{meta}}_{\hat{G}}$.

rather than running to full convergence — and then backpropagating through these intermediate updates. Specifically, we compute the meta-gradient $\nabla^{\text{meta}}_{\hat{G}}$ of the outer loss $\mathcal{L}_{\text{gen}}$ with respect to the hyperparameter $\hat{G}$:

$$\begin{aligned} \nabla^{meta}_{\hat{G}} &= \nabla_{\hat{G}} \mathcal{L}_{gen}\left(\hat{f}(\hat{G}; \theta^*), Y_P\right) \\ \text{s.t.} \quad \theta^* &= \arg\min_{\theta} \mathcal{L}_{train}\left(\hat{f}(\hat{G}; \theta), Y_L\right). \end{aligned} \tag{3}$$

The meta-gradient indicates how the graph generation loss $\mathcal{L}_{gen}$ will change after training on the new graph. For simplicity, we use $\mathcal{L}_{gen}\hat{f}_{\theta^*}(\hat{G})$ to denote $\mathcal{L}_{gen}(\hat{f}(\hat{G}; \theta^*), Y_P)$, and $\mathcal{L}_{train}\hat{f}_{\theta}(\hat{G})$ to denote $\mathcal{L}_{train}(\hat{f}(\hat{G}; \theta), Y_L)$, in the following discussions.

The optimization of the inner-level objective in Eqn. (2) can be achieved by multiple steps of gradient descent starting from the initial parameter θ_0 with the learning rate η:

$$\theta_{i+1} = \theta_i - \eta \nabla_{\theta_i} \mathcal{L}_{train}\hat{f}_{\theta_i}(\hat{G}). \tag{4}$$

Then the meta-gradient at the i-th iteration can be approximated as the follows (illustrated in Figure 3):

$$\begin{aligned} \nabla^{\text{meta}}_{\hat{G}} =& \frac{\partial \mathcal{L}_{\text{gen}}\left(\hat{f}_{\theta_i}(\hat{G})\right)}{\partial f}\left[\frac{\partial \hat{f}_{\theta_i}(\hat{G})}{\partial \hat{G}} + \frac{\partial \hat{f}_{\theta_i}(\hat{G})}{\partial \theta_i}\frac{\partial \theta_i}{\partial \hat{G}}\right] \\ =& \nabla_{\hat{f}} \mathcal{L}_{\text{gen}}\left(\hat{f}_{\theta_i}(\hat{G})\right)\left[\nabla_{\hat{G}} \hat{f}_{\theta_i}(\hat{G}) + \nabla_{\theta_i} \hat{f}_{\theta_i}(\hat{G}) \nabla_{\hat{G}} \theta_i\right] \end{aligned} \tag{5}$$

where

$$\nabla_{\hat{G}} \theta_i = \nabla_{\hat{G}} \theta_{i-1} - \eta \nabla_{\hat{G}} \nabla_{\theta_{i-1}} \mathcal{L}_{train}(\hat{f}_{\theta_{i-1}}(\hat{G})). \tag{6}$$

5.3 Graph Reconstruction through Meta-gradient

We construct the graph $\hat{G}$ in an incremental fashion. Specifically, we initialize $\hat{G}$ as the graph that contains all nodes in G but no edges (i.e., all nodes are disconnected in $\hat{G}^0$). Then at each iteration of the optimization of the inner-level objective in Eqn. (2), we update the adjacency matrix $\hat{A}_{i+1}$ of $\hat{G}$ as follows:

$$\hat{A}_{i+1} = \hat{A}_i - \eta \cdot \nabla^{\text{meta}}_{\hat{G}} \tag{7}$$

One problem of Eqn. (7) is that $\hat{A}_{i+1}$ is not a binary matrix. To fix this problem, first, we compute the complementary adjacency matrix $\bar{A} = J - A$, where J is the all-ones matrix, and A is the adjacency matrix of G. Therefore, $\bar{A}[j,k] = 1 - A[j,k]$ for each entry $A[j,k]$. Next, we compute a *score matrix* $g = \nabla^{meta}_{\hat{G}} \odot \bar{A}$, obtained by performing element-wise production between the meta-gradients and $\bar{A}$. This operation ensures that the node pairs that are connected in G will have score 0 in g. Then we greedily pick

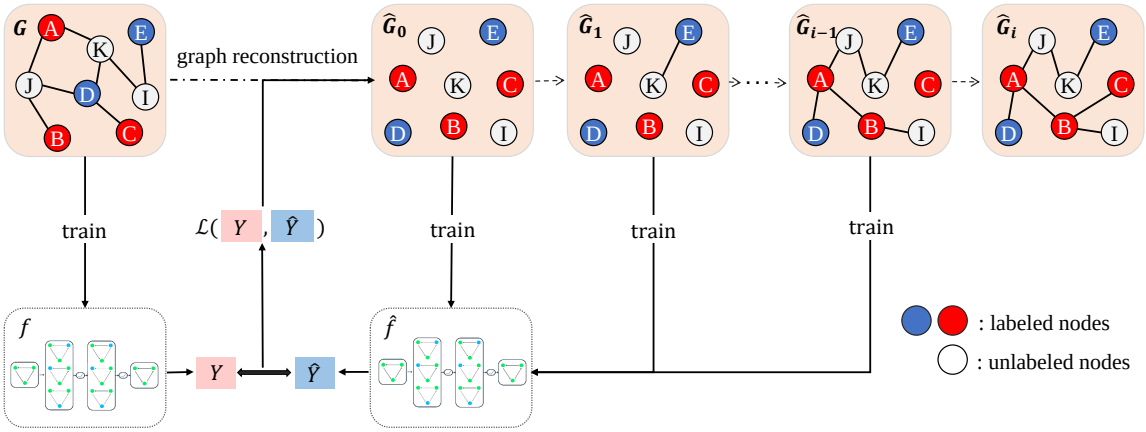


Figure 4: Illustration of PGR. In both G and $\hat{G}$, nodes A - E are labeled, with colors representing their respective classes.

Algorithm 1: PGR algorithm

Input: G: Input graph; A: the adjacency matrix of G; f: GNN model trained on G; $\hat{K}$: number of edges in the constructed graph $\hat{G}$.

1 $Y_L \leftarrow$ Labels of the labeled nodes in G;
2 $Y_P \leftarrow$ Labels of unlabeled nodes in G predicted by f;
3 $\bar{A} = J - A$;
4 $\hat{A}_0 \leftarrow$ A self-connected diagonal matrix;
5 $\theta_0 \leftarrow$ Obtained by training on $\hat{A}_0$;
6 **for** $i \in 0 \ldots \hat{K} - 1$ **do**
7 $\quad \theta_{i+1} = \theta_i - \eta \nabla_{\theta_i} \mathcal{L}_{train}(\hat{f}_{\theta_i}(\hat{G}), Y_L)$;
8 $\quad \mathbf{g}_i \leftarrow (\nabla_{\hat{G}} \mathcal{L}_{gen}(\hat{f}_{\theta_{i+1}}(\hat{G}), Y_P) \odot \bar{A}$;
9 $\quad (j, k) \leftarrow$ The index of the smallest gradient in $\mathbf{g}_i$;
10 $\quad \hat{A}_i[j, k] = 1$;
11 **return** $\hat{G}, \theta_{\hat{K}}$.

the nodes j, k such that $g[j, k]$ has the lowest non-zero score in g:

$$e^* = \arg\min_{e(j,k) \notin \hat{A}_i} g[j, k] \tag{8}$$

Then we insert the edge $e(j, k)$ into $\hat{G}$ by changing $\hat{A}_{i+1}[j, k] = 1$. We choose this node pair as the insertion of its corresponding edge will minimize the loss on node classification. This completes the process of inserting one edge. The algorithm terminates after $\hat{K}$ iterations, where $\hat{K}$ is a user-specified parameter for the number of edges in $\hat{G}$. It guarantees that the reconstructed graph $\hat{G}$ consists of $\hat{K}$ edges which do not appear in G.

5.4 Implementation and Optimization

Putting everything together, Algorithm 1 sketches the flow of PGR. At a high level, PGR is implemented as an iterative process (as illustrated in Figure 4). Specifically, PGR trains a GNN f on G, and obtains the labels Y_P predicted by f (Line 2). Next, it initializes $\hat{G}_0$ as a graph that contains nodes only (Line 4). At the i-th iteration, PGR reconstructs a new graph $\hat{G}_i$ using the meta-gradients (Lines 7 - 10) to insert one single edge. Therefore, PGR inserts $\hat{K}$ edges by repeating $\hat{K}$ iterations.

Efficient computation of meta-gradient. We compute the meta-gradient using the built-in differentiation engine provided by PyTorch [24] for computation of differentiation and gradients.

One-step truncated back-propagation. A commonly used approximate solution of meta-learning is *k-step truncated back-propagation* [29], by which the inner model parameters θ is updated for k steps rather than running to full convergence. In our implementation, we consider 1-step truncated back-propagation, i.e., Algorithm 1 performs a single iteration to update $\hat{\theta}_{i+1}$ (Line 7). Our empirical results (Section 6.4) will demonstrate that one-step truncated back-propagation is sufficient to effectively guide the optimization of the outer objective function of PGR.

5.5 DP-PGR: Equip PGR with Edge-level DP

Although PGR significantly mitigates the topology privacy risks of GNNs against TIAs (see Section 6), it has not provided formal privacy guarantee, even at the edge level. Therefore, in this section, we discuss how to equip PGR with the edge-level DP guarantee. Note that our goal is not to design new DP algorithms. Instead, we aim to show that PGR can be seamlessly integrated with existing edge-level DP methods and offer privacy protection at both edge and topology levels.

First, we formalize the definition of edge-level DP.

Definition 5.1. (**Edge-level Differential privacy [17]**). *A randomized algorithm $\mathcal{A}$ satisfies (ϵ, δ)-edge differential privacy, if for any two adjacent graphs G and G' that differs in a single edge, and for any possible set of outputs $S \subseteq Range(\mathcal{A})$, we have:*

$$\forall \Pr[\mathcal{A}(G) \in S] \leq e^{\epsilon} \Pr[\mathcal{A}(G') \in S] + \delta.$$

Here, ϵ is a non-negative parameter that quantifies the privacy loss, with smaller values indicating stronger privacy guarantees. The parameter $\delta \geq 0$ is a small probability that allows for a relaxation of the strict ϵ-differential privacy condition. When $\delta = 0$, (ϵ, δ)-DP becomes ϵ-DP.

In general, existing edge-DP approaches can be broadly divided into two categories: 1) *Graph synthesis (GS)* methods [31, 37, 39] :

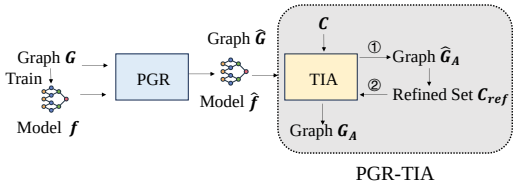


Figure 5: Illustration of PGR-TIA.

these methods generate a synthesized graph that satisfies edge-DP. Then the GNN model is trained on the synthesized graph. 2) *Noisy aggregation (NAG)* methods [19, 28]: these methods add noise to feature aggregation during model training.

Due to the fundamental difference between these two categories, we adapt the two categories of edge-DP approaches to PGR in different ways:

GS-based edge-DP methods: To equip edge-DP with PGR, first, we replace Y_P of Algorithm 1 (Line 2) with Y_P^{DP}, i.e., the labels output by the differentially private GNN models. Second, we replace A in Algorithm 1 (Line 3) with A^{DP}, where A^{DP} is the adjacency matrix of the graph generated by the GS-based methods.

NAG-based edge-DP methods: Similar to the GS-based methods, Y_P of Algorithm 1 (Line 2) will be replaced with Y_P^{DP}. However, as these methods do not generate a graph that satisfies edge-DP, we cannot replace A in Algorithm 1 (Line 3) with a graph that satisfies edge-DP. Consequently, we cannot compute g_i from the complementary graph (Line 8). Instead, we pick the smallest gradient from $\mathbf{g}_i = (\nabla_{\hat{G}} \mathcal{L}_{gen}(\hat{f}_{\theta_{i+1}}(\hat{G}), Y_P)$. Doing this cannot guarantee that $\hat{G}$ is disjoint from G, thus bringing potential downgrade in topology privacy protection. However, this is the price we have to pay for achieving edge-level DP.

We have the following theorem.

Theorem 5.2. *DP-PGR satisfies (ϵ, δ) edge-DP.*

The proof of Theorem 5.2 can be found in Appendix C. To summarize, the design of DP-PGR shows that PGR can be easily integrated into the existing edge-DP methods, providing privacy protection at both the edge and topology levels.

5.6 Robustness of PGR against Grey-box Adversary

In this section, we discuss the robustness of PGR in the presence of adversaries who try to bypass it. We consider a *grey-box adversary* who has not only black-box access to the trained GNN model $\hat{f}$ but also some partial knowledge of PGR, including its mechanisms and the value of its parameters such as $\hat{K}$. The adversary will exploit the knowledge of PGR to enhance the TIA accordingly. Following this intuition, we design **PGR-TIA**, an enhanced TIA against PGR. Next, we present the details of PGR-TIA.

Notably, PGR possesses a key property: *the training graph $\hat{G}$ used for the output model $\hat{f}$ shares no edges with the original training graph G used to train f*. This property enables a grey-box adversary to design stronger TIAs. PGR-TIA is tailored to exploit this edge-disjointness information. Importantly, PGR-TIA does not require access to the original graph G or model f - it operates solely via black-box access to $\hat{f}$.

Specifically, let G denote the original graph and f the GNN trained on it. Let $\hat{G}$ and $\hat{f}$ represent the graph and model generated by PGR, respectively. Define the set of all possible node pairs (i.e., candidate edges) in G as $C = \{(u, v) \mid (u, v) \in G\}$. Essentially, PGR-TIA consists of two rounds of TIAs. In the first round, the adversary performs a black-box attack on $\hat{f}$ to infer a graph $\hat{G}_A$, with $\hat{E}_A$ denoting its edge set. Since the adversary knows that it is highly likely that $\hat{E}_A \cap E = \emptyset$, they can refine the candidate edge set C by excluding the edges inferred in $\hat{G}_A$. That is, the refined candidate set is defined as $C_{\text{ref}} = C \setminus \hat{C}_A$, where $\hat{C}_A$ includes all node pairs connected in $\hat{G}_A$. Using this refined set, the adversary then conducts the second round of TIA, selecting edges exclusively from Cref to infer G_A.

Compared to traditional TIAs, PGR-TIA reduces the edge search space from $\frac{N(N-1)}{2}$ to $\left(\frac{N(N-1)}{2} - \hat{K}\right)$, where $\hat{K}$ denotes the number of edges in $\hat{G}$ - a tunable parameter in PGR. Intuitively, a larger $\hat{K}$ offers stronger pruning power, leading to a more efficient attack. However, the effectiveness of this refinement depends heavily on the accuracy of the TIA. The pruned edges are guaranteed to be correct only if $\hat{G}_A = \hat{G}$, i.e., the TIA achieves perfect accuracy—an unrealistic assumption in practice. When the TIA is imperfect, some edges in $\hat{E}_A$ may still exist in E, resulting in true edges being incorrectly excluded from the refined candidate set. These exclusions can degrade the final accuracy of the inferred graph G_A. Intuitively, a smaller $\hat{K}$ leads to a higher likelihood of such erroneous exclusions. For this reason, we prefer a smaller $\hat{K}$, as it enhances PGR's robustness against PGR-TIA.

6 Evaluation

In this section, we evaluate the performance of PGR by answering the following five research questions:

- **RQ$_1$** - How effective is PGR against TIAs?
- **RQ$_2$** - How does PGR perform compared to the existing edge-DP approaches in terms of the trade-off between topology privacy protection and model accuracy?
- **RQ$_3$** - How do various parameters impact PGR's performance?
- **RQ$_4$** - How robust is PGR against the PGR-TIA attack?
- **RQ$_5$** - How does DP-PGR perform compared to existing edge-DP approaches in terms of both topology privacy protection and model accuracy?

6.1 Experiment Setup

All the experiments are executed on a server with NVIDIA A100 GPU and 40GB memory. All the algorithms are implemented in Python along with PyTorch. All results were evaluated as the average of three trials.

Datasets. Our experiments utilize four real-world graph datasets, including two citation graphs (Cora [38] and Citeseer [38]) and two social network graphs (LastFM [27] and Emory [32]). The statistics of these datasets are shown in Table 3. For each dataset, 10% of nodes are randomly selected as the labeled nodes for training, and the remaining 90% nodes are used for testing.

GNN models. We consider three mainstream GNN models, namely, Graph Convolutional Networks (GCN) [32], Graph Attention Networks (GAT) [25], and GraphSAGE [10]. We set up both GCN and GraphSAGE as two-layer neural networks with a

Table 3: Statistics of datasets

Type	Dataset	Nodes	Edges	Density	Attrs	Classes
Citation Network	Cora	2708	5429	0.14%	1433	7
	Citeseer	3327	4732	0.08%	3703	6
Social Network	LastFM	7624	55612	0.19%	128	18
	Emory	5125	425138	3.23%	288	5

32-neuron hidden layer, and GAT as a 2-layer network with an 8-neuron hidden layer. We utilize the ReLU activation function and Softmax function for the output. We employ the Adam optimizer for training, with a learning rate of 0.01, and a weight decay of $5e-4$. All the three models were trained by 100 epochs.[8]

Evaluation metrics. We consider following evaluation metrics:

- *Topology privacy vulnerability:* We utilize TPL (Eqn. (1)) to evaluate the effectiveness of the defense mechanism. A higher (lower, resp.) TPL value indicates weaker (stronger, resp.) defense.
- *Model accuracy loss:* We measure the *loss of model accuracy* as $\frac{ACC'-ACC}{ACC}$, where ACC and ACC' are the accuracy of node classification before and after the deployment of the defense. The model accuracy loss can be either positive or negative: a positive (negative, resp.) loss indicates that the node classification accuracy is downgraded (upgraded, resp.) by the privacy protection mechanism.

Methods for comparison. We compare PGR with six existing graph privacy protection approaches: Eclipse [31], LPGNet [19], PrivGraph [39], GAP [28], LapEdge [37] and EdgeRand [37].[9] Among these methods, Eclipse, PrivGraph, LapEdge, and EdgeRand generate synthetic graphs for training the GNN model, while LPGNet and GAP perturb the GNN training process. We use the privacy budget $\epsilon \in \{1, 3, 5, 7, 9\}$ for these approaches.

6.2 Effectiveness of PGR (RQ_1)

Table 4 presents the TPL results against all three TIAs, both before and after the deployment of PGR. Overall, PGR significantly reduces topology privacy leakage across all attacks, datasets, and models, demonstrating its effectiveness as a defense mechanism. Notably, PGR is particularly effective against I-TIA - the strongest of the three attacks - reducing TPL from approximately 100% to no more than 5% for both Cora and Citeseer datasets, and around 10% for the Emory dataset. For M-TIA and C-TIA, although the reductions in TPL are less dramatic compared to I-TIA, PGR still achieves substantial mitigation. One exception is the Emory dataset, where PGR does not lower the TPL to the same extent as in other datasets. However, this is acceptable, as the resulting TPL in this case is already close to the level of random guessing. The variation in PGR's defense effectiveness across datasets can be attributed to differences in graph density, which we further analyze in Section 6.4.

Additionally, while offering strong protection for topology privacy, PGR incurs minimal loss in model accuracy, with the maximum accuracy degradation not exceeding 0.87%. In rare cases, we even observe that PGR slightly outperforms the original model by 0.12% in accuracy.

In summary, PGR provides topological privacy protection while preserving the model performance, addressing the insufficiency of existing edge-DP approaches.

6.3 Trade-off between Privacy and Model Accuracy (RQ_2)

Privacy often comes at the expense of reduced model utility. To illustrate the trade-off between privacy and model accuracy, we construct a TPL-utility curve, where each point represents a pair of topology privacy leakage (TPL) and node classification accuracy values. This curve is generated by varying the parameter $\hat{K}$ of PGR as well as the privacy budget ϵ of the baselines, and measuring both the TPL and node classification accuracy for each parameter setting. Intuitively, a method that achieves lower TPL and higher node classification accuracy strikes a better balance between privacy and model performance.

Figure 6 demonstrates the TPL-utility curve of PGR and the six existing edge-DP approaches on four datasets. Intuitively, the defense method that has higher defense strength (i.e. lower TPL) and aligned target model accuracy with the others has a better trade-off between privacy and model accuracy. We observe that PGR outperforms the six baselines in the trade-off between defense effectiveness and model accuracy: PGR achieves lower TPL than the baselines under the same target model accuracy, while it delivers higher target model accuracy than the baselines when they have the same attack accuracy.

6.4 Factor Analysis (RQ_3)

In this part of the experiment, we investigate the impact of several key factors on the performance of PGR, including: (1) the number of edges $\hat{K}$ in the reconstructed graph $\hat{G}$, which is a tunable parameter in PGR; (2) the structural characteristics of the original training graph G; and (3) the number of iterations used for optimizing the inner objective function in Algorithm 1. We report the highest TPL observed across the three TIAs, along with the corresponding model accuracy. Additionally, we evaluate the effect of varying the number of GNN layers on PGR's performance, with detailed results provided in Appendix D.1.

Number of edges $\hat{K}$. Figure 7 shows the results of both topology privacy leakage (TPL) and model accuracy under varying values of $\hat{K}$, the number of edges in the reconstructed graph. Instead of fixing $\hat{K}$, we express it as a proportion of K, the number of edges in the original training graph G. The results reveal that TPL remains relatively stable as $\hat{K}$ increases, highlighting the robustness of PGR's defense capability. In contrast, model accuracy improves with larger $\hat{K}$. Notably, PGR does not require a large $\hat{K}$ to achieve strong performance. Even when $\hat{G}$ contains significantly fewer edges than G, it can still support effective node classification. For example, on the Citeseer dataset, a reconstructed graph with only 30% of G's edges achieves comparable accuracy. This efficiency is

[8]Our empirical results show that 100 epochs are sufficient, as the model can achieve satisfactory accuracy. Specifically, with GCN model, the model's accuracy on the Cora, Citeseer, LastFM, Emory, and Facebook datasets is 83.38%, 70.31%, 81.75%, 90.68%, and 91.08%, respectively. For GTA, we train 200 epochs on Emory.

[9]We do not consider PPNE [11] and PrivGNN [21] as both PPNE and PrivGNN assume the availability of either a partial graph or an external dataset, which do not apply to our setting.

Table 4: Topology privacy leakage (TPL) and model accuracy loss of PGR. We consider three TIAs for measuring TPL. A positive (negative, resp.) loss indicates that the node classification accuracy is downgraded (upgraded, resp.) by privacy protection.

Model	Dataset	Topology privacy leakage (%)						Model accuracy loss (%)
		Original model			PGR			
		M-TIA	C-TIA	I-TIA	M-TIA	C-TIA	I-TIA	
GCN	Cora	28.10	25.53	100.00	3.01	2.70	0.81	0.00
	Citeseer	17.33	10.73	100.00	4.17	2.30	1.14	0.00
	Emory	17.55	16.54	100.00	12.53	12.01	9.32	0.38
GAT	Cora	14.85	11.50	100.00	2.71	2.44	1.08	-0.12
	Citeseer	20.45	11.66	100.00	4.81	4.22	3.51	0.00
	Emory	16.76	17.44	100.00	12.03	11.34	10.44	0.43
GraphSAGE	Cora	13.13	9.68	83.15	4.10	2.40	4.74	0.13
	Citeseer	17.57	17.75	100.00	4.50	4.20	3.51	0.00
	Emory	18.79	17.62	94.86	13.21	12.45	10.10	0.87

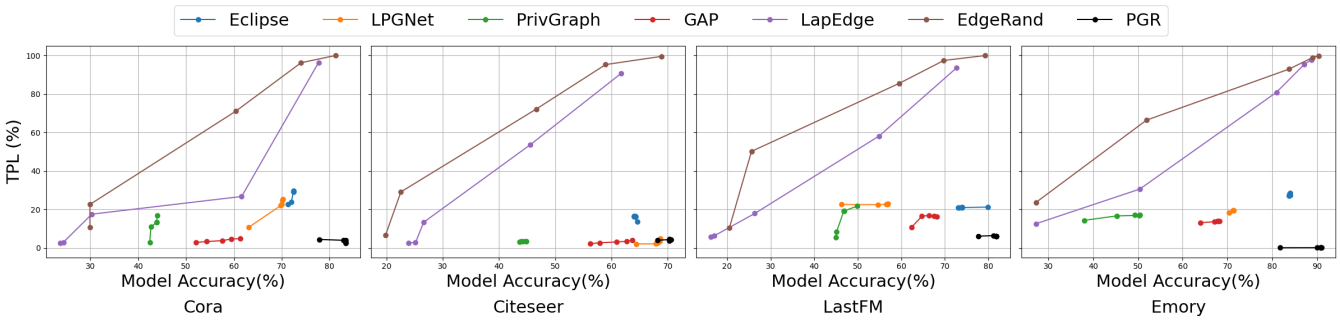


Figure 6: The trade-off between topology privacy leakage and model accuracy. The x-axis represents node classification accuracy (%), while the y-axis represents topology privacy leakage (TPL). For all edge-dp methods, we set the privacy budget ϵ in {1,3,5,7,9}.

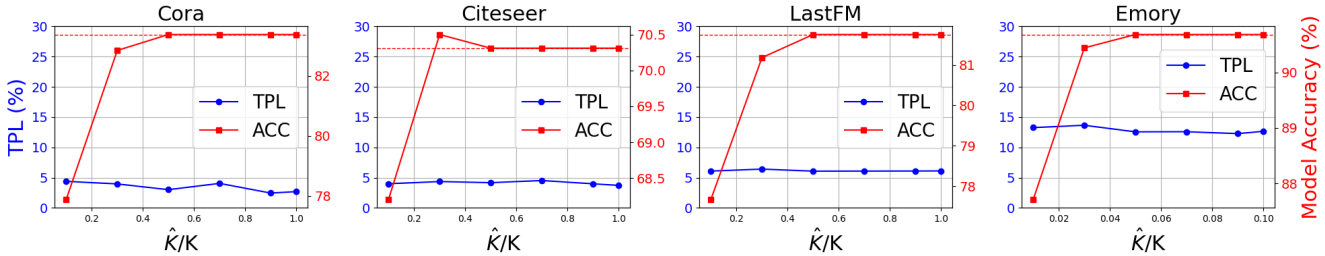
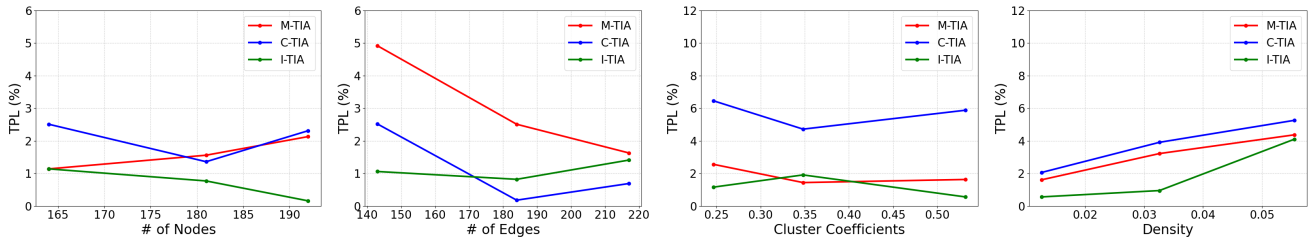


Figure 7: Impact of the number of edges $\hat{K}$ on the performance of PGR. The x-axis indicates the proportion $\hat{K}/K$ (K: number of real edges). The left (blue) y-axis indicates the topology privacy leakage (TPL), while the right (red) y-axis indicates model accuracy. The dotted red line denotes the accuracy of the original model without privacy protection.

even more pronounced on denser graphs—on the Emory dataset, just 3% of G's edges are sufficient to maintain similar performance.

Graph characteristics. In our previous results, we observed that PGR's performance varies across different graphs. To better understand how specific graph characteristics influence its effectiveness, we investigate four key graph measures: *number of nodes*, *number of edges*, *density*, and *clustering coefficient*. To isolate the impact of each factor, we randomly sample subgraphs from the Cora dataset, keeping the number of nodes fixed while varying the other three properties. Figure 8 presents the TPL results of PGR under these settings. The TPL does not exhibit consistent changes when the three types of graph measures — edge count, node count, and clustering coefficient— are varied. This suggests that PGR remains effective across datasets of varying sizes and community structures. In contrast, we observe a consistent pattern in TPL changes as graph density increases (Figure 8 (d)). Specifically, PGR performs better on sparser graphs, exhibiting lower TPL values. This aligns with our earlier observation that PGR achieves the highest TPL on the Emory dataset, which is the densest among all the datasets.

One-step truncated back-propagation optimization. Recall that we use the one-step truncated back-propagation to approximate the parameters (Section 5.4). In this part of experiments, we

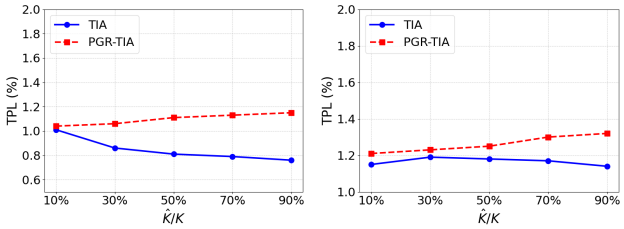


(a) Number of nodes (b) Number of edges (c) Cluster coefficient (d) Graph density

Figure 8: The impact of graph characteristics on the performance of PGR (Cora dataset).

Table 5: Impact of one-step truncated back-propagation on PGR's performance. The highest TPL among the three TIAs is reported.

Setting	Cora		Citeseer		LastFM		Emory	
	ACC	TPL	ACC	TPL	ACC	TPL	ACC	TPL
Conv.	83.38	3.21	70.31	3.62	81.75	6.14	91.07	12.57
1 iter.	83.38	3.01	70.31	4.17	81.75	6.07	90.75	12.53



(a) Cora (b) Citeseer

Figure 9: Topology privacy leakage (%) of PGR against both TIA and PGR-TIA.

evaluate the impact of this optimization strategy on PGR's performance. We define convergence as the condition where the change in the loss remains within 0.01 over five consecutive iterations. Table 5 compares the performance of PGR under two settings: (1) performing only a single update iteration for the inner optimization loop (Line 9 of Algorithm 1), and (2) training the model until the inner objective converges. The results reveal that PGR achieves comparable performance in both settings—model accuracy and TPL remain similar regardless of whether a single iteration or full convergence is used. This validates our design choice and highlights the efficiency of the proposed optimization approach in Algorithm 1.

6.5 Robustness of PGR against PGR-TIA (RQ_4)

To assess the robustness of PGR against PGR-TIA, we compare its topology privacy leakage (TPL) under both the original TIAs and their corresponding PGR-TIA variants. Recall that the filtering capability of PGR-TIA depends on the value of $\hat{K}$ (Section 5.6), which represents the number of edges in the reconstructed graph. Accordingly, we vary $\hat{K}$ from 10% to 90% of the original graph's size to evaluate different attack strengths. As shown in Figure 9, PGR experiences only a marginal increase in TPL - no more than 0.4% - when subjected to PGR-TIA compared to the original TIAs, across all settings. These results highlight the robustness of PGR in preserving topological privacy, even under more informed and adaptive attacks.

6.6 Performance of DP-PGR (RQ_5)

To evaluate the performance of DP-PGR, we deploy Eclipse [31] on top of PGR (named Eclipse-PGR), using the privacy budget $\epsilon = \{1, 5, 9\}$. The results of DP-PGR based on other edge-DP approaches can be found in Appendix D.2.

Figure 10 presents the TPL and model accuracy results for both Eclipse and Eclipse-PGR. First, with the addition of edge-level DP protection, Eclipse-PGR achieves significantly better topology privacy than PGR, showing a substantial reduction in TPL across all four datasets. Second, Eclipse-PGR maintains model accuracy comparable to that of PGR in all settings. These results demonstrate that PGR can be seamlessly integrated with existing edge-level DP methods to enhance topology privacy protection without compromising model performance.

7 Discussions

7.1 Scalability of TIA and PGR

So far, our evaluation of TIAs and PGR has focused on small subgraphs. But how do these methods perform when applied to GNN models trained on full, large-scale graphs? To answer this, we consider the entire graph as the attack target and assess the performance of both TIAs and PGR. As shown in Table 6, both TIAs and PGR remain highly effective at scale. Specifically, I-TIA continues to achieve a TPL of 100%, while M-TIA and C-TIA yield TPLs ranging from 6.22% to 11.08%. In contrast, PGR successfully reduces TPL for I-TIA to just 0.04%–0.13%, and brings M-TIA and C-TIA TPLs below 4% in all cases.

However, for extremely large graphs, both TIA and PGR may require substantial computational resources, which may not always be feasible. To address this scalability challenge, the graph can be partitioned into smaller subgraphs, allowing TIAs to be executed independently on each part. The inferred subgraphs can then be merged into a unified graph structure. PGR can adopt a similar strategy: it constructs multiple synthetic subgraphs $\hat{G}_1, \ldots, \hat{G}_t$, each with an edge set disjoint from the original graph G. These subgraphs are then stitched together to form a global synthetic graph $\hat{G} = \cup_{i=1}^{t} \hat{G}_i$, on which a new GNN model $\hat{f}$ is trained.

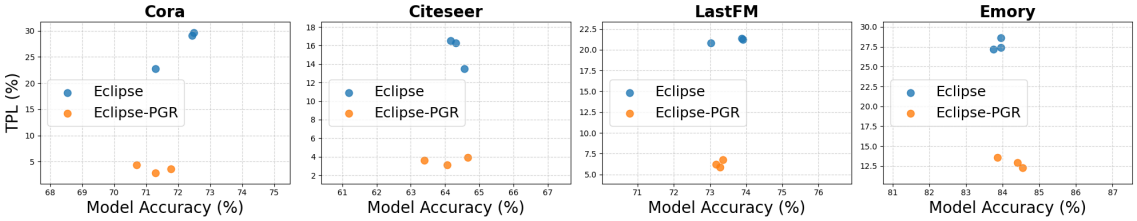


Figure 10: Trade-off between TPL and model accuracy of DP-PGR, with Eclipse [31] as the edge-DP approach

Table 6: Performance of TIAs and PGR on large graphs

Dataset	Without defense			PGR		
	M-TIA	C-TIA	I-TIA	M-TIA	C-TIA	I-TIA
Cora	10.8	8.65	100.00	0.91	0.27	0.13
Citeseer	11.08	9.76	100.00	0.57	0.26	0.06
Emory	7.06	6.22	100.00	3.91	3.62	0.04

Table 7: Accuracy of graph property inference by TIA (GCN model, Cora dataset). The accuracy of community detection is measured using the Adjusted Rand Index (ARI), and the accuracy of influential node detection is assessed using the F1-score.

Attack	Community detection	Detection of influential nodes	
		Degree-based	Centrality-based
M-TIA	0.15	0.16	0.18
C-TIA	0.13	0.13	0.19
I-TIA	1.00	1.00	1.00

7.2 From Topology Inference to Graph Property Inference

Since the topology of graphs can reveal critical structural information, an important question arises: can TIAs also expose key properties of the training graph? To investigate this, we evaluate the effectiveness of TIAs in inferring two fundamental graph properties: *community structures* and *influential nodes*.

- **Community detection:** We use Louvain [2], a SOTA community detection algorithm, on both the private graph and the attack graph. We use the Adjusted Rand Index (ARI) [7] (ranging in [-1, 1]) to measure the similarity between the communities detected in each graph. A higher ARI indicates a higher accuracy of community detection from the attack graph.
- **Influential nodes:** We define *influential nodes* as the top 100 nodes ranked by degree and by centrality, respectively. We measure the precision and recall of the influential nodes identified in the attack graph, and report the F1-score computed from the precision and recall.

Table 7 presents the results for the GCN model trained on the Cora dataset. All three TIAs successfully infer communities and influential nodes from the model, with I-TIA achieving the highest accuracy in both tasks. These results demonstrate that TIAs, particularly I-TIA, can reveal community structures and influential nodes in the private graph, exposing privacy risks that extend beyond simple link disclosure.

Table 8: Mitigation of edge-level privacy vulnerability by PGR. The accuracy (in %) of link membership inference attacks is reported.

Model	Dataset	Without defense		PGR	
		StealLink	Infiltrator	StealLink	Infiltrator
GCN	Cora	84.86	100.00	66.10	49.74
	Citeseer	88.12	100.00	61.17	50.98
	Emory	62.93	100.00	50.54	49.20
GAT	Cora	85.16	91.72	63.97	49.74
	Citeseer	86.81	98.62	63.23	50.98
	Emory	60.62	74.05	51.21	50.12
GraphSAGE	Cora	81.15	97.02	65.98	49.74
	Citeseer	90.30	100.00	61.17	50.98
	Emory	63.86	78.44	50.98	49.98

7.3 Mitigation of Edge-level Privacy Vulnerability by PGR

We have demonstrated that PGR effectively mitigates topology-level privacy leakage in GNNs. But can it also defend against edge-level privacy attacks? To answer this question, we evaluate PGR's resilience against the *link membership inference attacks* (LMIAs). Specifically, we consider two state-of-the-art LMIA methods: *StealLink* [14] and *Infiltrator* [20]. To evaluate the accuracy of these attacks, we randomly sample 500 connected node pairs (members) and 500 unconnected node pairs (non-members) to form the test set. We measure attack accuracy as the proportion of correctly identified member and non-member edges. We repeat over five independent trials, and report the average results.

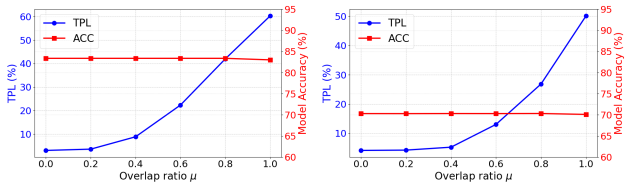
Table 8 shows the attack accuracy of StealLink and Infiltrator against the original GNN model and the model protected by PGR. The results reveal a clear trend: PGR significantly reduces the effectiveness of both attacks, with Infiltrator's accuracy dropping to nearly 50% — equivalent to random guessing. These findings highlight that PGR not only strengthens topology-level privacy but also provides strong protection against edge-level membership inference threats.

7.4 Alternative Topology Privacy Evaluation Metric

TPL is primarily measured by the Jaccard similarity between the original and reconstructed graphs. Are there alternative metrics for evaluating topology privacy leakage? To answer this question, we design the following metric that assess topology privacy leakage through the *precision* and *recall* of the edges in the attack graph. Specifically, let E_T and E_A be the edges in the private graph and the attack graph, respectively. The precision is measured as $\frac{|E_T \cap E_A|}{|E_A|}$,

Table 9: Topology privacy leakage of TIAs measured by Jaccard similarity (Sim.) and F1-score (GCN model, Cora dataset).

Method	M-TIA		C-TIA		I-TIA	
	F1-score	Sim.	F1-score	Sim.	F1-score	Sim.
W/O PGR	42.12	28.10	37.69	25.53	100.00	100.00
With PGR	6.20	3.01	5.34	2.70	1.89	0.81



(a) Cora (b) Citeseer

Figure 11: Impact of overlapping ratio μ on PGR's performance. The left (blue) and the right (red) y-axes indicate topology privacy leakage and model accuracy, respectively.

and recall is measured as $\frac{|E_T \cap E_A|}{|E_T|}$. Finally, we compute the F1-score from the precision and recall.

Table 9 presents the TPL results, measured by Jaccard similarity and F1-score, before and after deploying PGR, with GCN trained on the Cora dataset as the target model. Both metrics yield consistent results across all settings, showing that I-TIA causes the highest privacy leakage, while C-TIA results in the lowest.

7.5 Relaxation of Non-overlapping Constraint of PGR

In the paper, we primarily focus on constructing a graph whose edges do not overlap with those in the original training graph. However, such a non-overlapping graph may not exist when G is extremely dense - for instance, in the case of a fully connected graph. To address this limitation, we relax the strict non-overlapping constraint by allowing a controlled level of overlap, defined as $\frac{|E \cap \hat{E}|}{|\hat{E}|} \leq \mu$, where $\mu \in [0, 1]$ is a user-defined parameter. This relaxed constraint can be easily incorporated into PGR by adding a simple check of the overlapping degree between E and $|\hat{E}|$ in Algorithm 1.

We evaluate the impact of the overlap constraint on PGR's performance by varying the overlap ratio μ from 0 to 1. We measure both the TPL and the model accuracy for these settings and presents the results on the Cora and Citeseer datasets in Figure 11. As μ increases, we observe a steady rise in TPL, indicating weaker privacy protection. However, the model accuracy remains largely unaffected, demonstrating PGR's robustness to edge overlap.

Notably, it is not easy to equip PGR-TIA with the overlapping constraint, as the edges in the inferred graph $\hat{G}$ may still exist in the original graph, making the adversary difficult to refine the candidate edge set for the second round of TIA. We will explore this scenario in the future work.

8 Conclusion

In this paper, we addressed the critical challenge of preserving topology privacy in GNNs. We first introduced a suite of black-box Topology Inference Attacks (TIAs), and showed that SOTA edge-DP methods are insufficient to defend against such attacks. To tackle this issue, we proposed PGR, a novel defense framework that effectively mitigates topology privacy leakage while maintaining high model accuracy. Extensive experiments across three widely-used GNN architectures and five real-world datasets demonstrate the strong protection and utility trade-off achieved by PGR.

For future work, we plan to investigate the privacy risks associated with other graph-level statistics, such as minimum and maximum node degrees, and their implications for topology inference. Additionally, we will explore new forms of adaptive attacks and evaluate the robustness of PGR against these more sophisticated threat models.

Acknowledgments

We thank the reviewers for their feedback. This work was supported by the National Science Foundation (CNS-2029038; CNS-2135988; CNS-2302689; CNS-2308730; CNS-2432533). Any opinions, findings, conclusions, or recommendations expressed in this paper are those of the authors and do not necessarily reflect the views of the funding agency. This work was initiated when Jie Fu was studying at East China Normal University with Prof. Zhili Chen.

References

[1] William Aiello, Fan Chung, and Linyuan Lu. 2000. A random graph model for massive graphs. In *Proceedings of the thirty-second annual ACM symposium on Theory of computing*. 171–180.
[2] Vincent D Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. Fast unfolding of communities in large networks. *Journal of statistical mechanics: theory and experiment* 2008, 10 (2008), P10008.
[3] Stacy Jo Dixon. 2025. "Number of Instagram users worldwide from 2019 to 2028 ". https://www.statista.com/forecasts/1138856/instagram-users-in-the-world.
[4] Vasisht Duddu, Antoine Boutet, and Virat Shejwalkar. 2020. Quantifying privacy leakage in graph embedding. In *MobiQuitous 2020-17th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services*. 76–85.
[5] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. 2006. Calibrating noise to sensitivity in private data analysis. In *Theory of Cryptography: Third Theory of Cryptography Conference, TCC 2006, New York, NY, USA, March 4-7, 2006. Proceedings 3*. 265–284.
[6] Cynthia Dwork, Aaron Roth, et al. 2014. The algorithmic foundations of differential privacy. *Foundations and Trends® in Theoretical Computer Science* 9, 3–4 (2014), 211–407.
[7] Kathleen M Gates, Teague Henry, Doug Steinley, and Damien A Fair. 2016. A Monte Carlo evaluation of weighted community detection algorithms. *Frontiers in neuroinformatics* 10 (2016), 45.
[8] Google. 2021. "Vertex ai - google cloud". https://cloud.google.com/vertex-ai.
[9] Zhiwei Guo and Heng Wang. 2020. A deep graph neural network-based mechanism for social recommendations. *IEEE Transactions on Industrial Informatics* 17, 4 (2020), 2776–2783.
[10] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).
[11] Xiao Han, Yuncong Yang, Leye Wang, and Junjie Wu. 2023. Privacy-preserving network embedding against private link inference attacks. *IEEE Transactions on Dependable and Secure Computing* (2023).
[12] Zhaoxing Han, Chengyu Hu, Tongyaqi Li, Qingqiang Qi, Peng Tang, and Shanqing Guo. 2024. Subgraph-level federated graph neural network for privacy-preserving recommendation with meta-learning. *Neural Networks* (2024), 106574.
[13] Haoyu He, Isaiah J King, and H Howie Huang. 2024. Maui: Black-Box Edge Privacy Attack on Graph Neural Networks. *Proceedings on Privacy Enhancing Technologies* (2024).

[14] Xinlei He, Jinyuan Jia, Michael Backes, Neil Zhenqiang Gong, and Yang Zhang. 2021. Stealing links from graph neural networks. In *30th USENIX security symposium*. 2669–2686.
[15] Chao Huang, Huance Xu, Yong Xu, Peng Dai, Lianghao Xia, Mengyin Lu, Liefeng Bo, Hao Xing, Xiaoping Lai, and Yanfang Ye. 2021. Knowledge-aware coupled graph neural network for social recommendation. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 4115–4122.
[16] IBM. 2018. "Infosphere virtual data pipeline — IBM". https://www.ibm.com/products/ibm-infosphere-virtual-data-pipeline.
[17] Vishesh Karwa, Sofya Raskhodnikova, Adam Smith, and Grigory Yaroslavtsev. 2011. Private analysis of graph structure. *Proceedings of the VLDB Endowment* 4, 11 (2011), 1146–1157.
[18] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
[19] Aashish Kolluri, Teodora Baluta, Bryan Hooi, and Prateek Saxena. 2022. LPGNet: Link private graph networks for node classification. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 1813–1827.
[20] Lingshuo Meng, Yijie Bai, Yanjiao Chen, Yutong Hu, Wenyuan Xu, and Haiqin Weng. 2023. Devil in Disguise: Breaching Graph Neural Networks Privacy through Infiltration. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 1153–1167.
[21] Iyiola Emmanuel Olatunji, Thorben Funke, and Megha Khosla. 2024. Releasing Graph Neural Networks with Differential Privacy Guarantees. *Transactions on Machine Learning Research* (2024).
[22] Iyiola E Olatunji, Wolfgang Nejdl, and Megha Khosla. 2021. Membership inference attack on graph neural networks. In *2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*. 11–20.
[23] Parlai. 2017. "Parlai". https://ai.facebook.com/tools/parlai.
[24] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
[25] Veli**v**cković Petar, Cucurull Guillem, Casanova Arantxa, Romero Adriana, Lio Pietro, and B Yoshua. 2018. Graph attention networks. In *International conference on learning representations*, Vol. 8.
[26] Yeqing Qiu, Chenyu Huang, Jianzong Wang, Zhangcheng Huang, and Jing Xiao. 2022. A privacy-preserving subgraph-level federated graph neural network via differential privacy. In *International Conference on Knowledge Science, Engineering and Management*. 165–177.
[27] Benedek Rozemberczki and Rik Sarkar. 2020. Characteristic functions on graphs: Birds of a feather, from statistical descriptors to parametric models. In *Proceedings of the 29th ACM international conference on information & knowledge management*. 1325–1334.
[28] Sina Sajadmanesh, Ali Shahin Shamsabadi, Aurélien Bellet, and Daniel Gatica-Perez. 2023. GAP: Differentially Private Graph Neural Networks with Aggregation Perturbation. In *32nd USENIX Security Symposium*. 3223–3240.
[29] Amirreza Shaban, Ching-An Cheng, Nathan Hatch, and Byron Boots. 2019. Truncated back-propagation for bilevel optimization. In *The 22nd International Conference on Artificial Intelligence and Statistics*. 1723–1732.
[30] Ruoxi Sun, Hanjun Dai, and Adams Wei Yu. 2022. Does gnn pretraining help molecular representation? *Advances in Neural Information Processing Systems* 35 (2022), 12096–12109.
[31] Tingting Tang, Yue Niu, Salman Avestimehr, and Murali Annavaram. 2024. Edge Private Graph Neural Networks with Singular Value Perturbation. *Proceedings on Privacy Enhancing Technologies* (2024).
[32] Amanda L Traud, Peter J Mucha, and Mason A Porter. 2012. Social structure of facebook networks. *Physica A: Statistical Mechanics and its Applications* 391, 16 (2012), 4165–4180.
[33] Guihong Wan and Harsha Kokel. 2021. Graph sparsification via meta-learning. (2021).
[34] Xiuling Wang and Wendy Hui Wang. 2023. Link membership inference attacks against unsupervised graph representation learning. In *Proceedings of the 39th Annual Computer Security Applications Conference*. 477–491.
[35] Xiuling Wang and Wendy Hui Wang. 2024. GCL-Leak: Link Membership Inference Attacks against Graph Contrastive Learning. *Proceedings on Privacy Enhancing Technologies* (2024).
[36] Xiuling Wang and Wendy Hui Wang. 2024. Subgraph Structure Membership Inference Attacks against Graph Neural Networks. *Proceedings on Privacy Enhancing Technologies* (2024).
[37] Fan Wu, Yunhui Long, Ce Zhang, and Bo Li. 2022. Linkteller: Recovering private edges from graph neural networks via influence analysis. In *2022 IEEE Symposium on Security and Privacy (SP)*. 2005–2024.
[38] Zhilin Yang, William Cohen, and Ruslan Salakhudinov. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*. 40–48.
[39] Quan Yuan, Zhikun Zhang, Linkang Du, Min Chen, Peng Cheng, and Mingyang Sun. 2023. PrivGraph: Differentially Private Graph Data Publication by Exploiting Community Information. In *32nd USENIX Security Symposium*. 3241–3258.
[40] Chenhan Zhang, Weiqi Wang, James JQ Yu, and Shui Yu. 2023. Extracting privacy-preserving subgraphs in federated graph learning using information bottleneck. In *Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security*. 109–121.
[41] Zhikun Zhang, Min Chen, Michael Backes, Yun Shen, and Yang Zhang. 2022. Inference attacks against graph neural networks. In *31st USENIX Security Symposium*. 4543–4560.
[42] Da Zhong, Ruotong Yu, Kun Wu, Xiuling Wang, Jun Xu, and Wendy Hui Wang. 2023. Disparate Vulnerability in Link Inference Attacks against Graph Neural Networks. *Proceedings on Privacy Enhancing Technologies* (2023).

Appendix

A TPL of Random Guessing

Table 10: TPL (%) of random guessing

	Cora	Citeseer	LastFM	Emory
Random guessing	1.66	1.64	4.27	10.09

Given a graph, let N and K be its number of nodes and edges, respectively, we randomly choose K node pairs from all possible pairs and construct edges between these pairs, producing the inferred graph G_A. We repeat this process for ten times and report the average TPL of all the trials in Table 10.

B Choice of Privacy Budget Value for Edge-DP Approaches

To show the trade-off between privacy and model accuracy of edge-DP approaches, we measure both topology privacy leakage and model accuracy (ACC) of the six SOTA edge-DP approaches under various privacy budget $\epsilon \in \{1, 3, 5, 7, 9\}$. Table 11 reports the model accuracy and TPL results across the four datasets. Among all the settings, $\epsilon = 7$ yields the best trade-off between model accuracy and privacy protection. This supports us to choose $\epsilon = 7$.

C Proof of Theorem 5.2

Following Algorithm 1, PGR requires access to the private graph G at two stages: **(i) Model training**: The model f is trained on G to generate the output Y_P (Line 2); **(ii) Graph overlap constraint**: The overlap between the generated graph $\hat{G}$ and G has to be evaluated at each iteration (Line 8). The challenge is to ensure edge-level DP guarantees for these two direct accesses to the private graph. For the model training phase, edge-DP methods can ensure that the output Y_p satisfies (ϵ, δ)-edge DP. For the graph overlap constraint phase, we choose to remove the constraint to avoid accessing the original graph. This allows us to leverage the post-processing property [6] of differential privacy to guarantee that DP-PGR satisfies (ϵ, δ)-edge DP.

D Additional Experimental Results

D.1 Impact of Number of GNN layers on Performance of PGR.

To evaluate the impact of the number of GNN layers on PGR's performance, we set up three GCNs with 1, 2, and 3 layers, respectively. There are 64 neurons at the first layer, 32 neurons at the second layer, and the number of neurons at the last layer equals the number of classes. We then measure both the TPL and model

Table 11: TPL and model accuracy (ACC) of six edge-DP approaches under various privacy budgets (ϵ).

Edge-DP approach	ϵ	Cora		Citeseer		LastFM		Emory	
		ACC	TPL	ACC	TPL	ACC	TPL	ACC	TPL
Eclipse	1	71.28	22.73	64.56	13.49	73.02	20.81	83.95	27.42
	3	72.01	23.90	63.99	16.14	73.28	20.91	83.82	28.21
	5	72.43	29.05	64.31	16.26	73.87	21.34	83.95	28.62
	7	72.45	29.50	63.99	16.51	73.87	20.94	83.78	27.36
	9	72.48	29.68	64.16	16.54	73.91	21.22	83.74	27.16
LPGNet	1	63.15	10.82	64.43	2.08	46.31	22.54	70.36	18.31
	3	69.80	21.99	67.84	2.11	54.69	22.46	71.16	19.51
	5	69.96	22.74	68.21	2.65	56.52	22.68	71.36	19.57
	7	70.13	24.67	68.57	3.53	56.82	22.74	71.38	19.38
	9	70.19	25.32	68.64	4.83	56.99	22.83	71.38	19.48
PrivGraph	1	42.43	2.95	43.62	3.13	45.01	5.58	38.05	14.35
	3	42.69	10.96	44.29	3.36	45.12	8.25	45.31	16.64
	5	43.86	13.23	44.76	3.51	46.79	19.11	50.29	16.91
	7	43.88	13.38	44.86	3.56	46.89	19.21	50.43	17.06
	9	43.97	16.77	43.92	3.39	49.92	21.74	49.28	17.12
GAP	1	52.15	2.84	56.24	2.22	62.35	10.74	63.99	13.11
	3	54.37	3.32	57.91	2.65	64.72	16.49	67.10	13.61
	5	57.61	3.78	60.92	3.09	66.38	16.87	67.71	13.89
	7	59.45	4.63	62.72	3.41	67.55	16.53	67.97	13.91
	9	61.39	4.79	63.71	4.09	68.19	16.34	68.21	13.97
LapEdge	1	23.76	2.43	23.95	2.59	16.25	5.90	27.19	12.54
	3	24.42	2.98	25.18	2.77	17.12	6.34	50.39	30.55
	5	30.34	17.53	26.62	13.31	26.37	17.89	80.92	80.84
	7	61.63	72.24	45.56	53.59	54.89	58.12	87.10	95.30
	9	77.70	96.35	61.69	90.61	72.70	93.61	88.73	97.78
EdgeRand	1	29.95	10.60	19.84	6.56	20.54	10.37	27.19	23.58
	3	29.95	22.59	22.58	29.10	25.62	50.16	51.89	66.37
	5	60.42	71.05	46.63	72.14	59.47	85.44	83.72	92.88
	7	74.04	96.22	58.95	95.25	69.75	97.39	89.05	98.98
	9	81.25	99.98	68.88	99.51	79.23	99.95	90.31	99.77

accuracy of these GNNs before and after deploying PGR. As shown in Figure 12, the number of GNN layers has minimal effect on both model accuracy and TPL. Specifically, the model accuracy of PGR remains consistently close to that of the original model, while TPL stays stable as the number of GCN layers increases, across all datasets. These findings demonstrate that PGR is robust and can be effectively deployed on GNNs of varying complexities without compromising either model performance or the protection of topology privacy.

D.2 Trade-off between Privacy and Model Accuracy of DP-PGR

Figure 13 reports the trade-off between TPL and model accuracy of the DP-PGR algorithm adapted from four edge-DP approaches (LPGNet [19], PrivGraph [39], LapEdge [37], and EdgeRand [37]). We report the best TPL result among the three TIAs for each dataset. We use the privacy budget $\epsilon \in \{1, 5, 9\}$ for all the algorithms and measure both TPL and model accuracy for each ϵ value. The results show that DP-PGR can equip PGR with DP effectively while minimizing the loss in model accuracy. This demonstrates that PGR can be seamlessly integrated with edge-DP while addressing the trade-off between privacy protection and model accuracy.

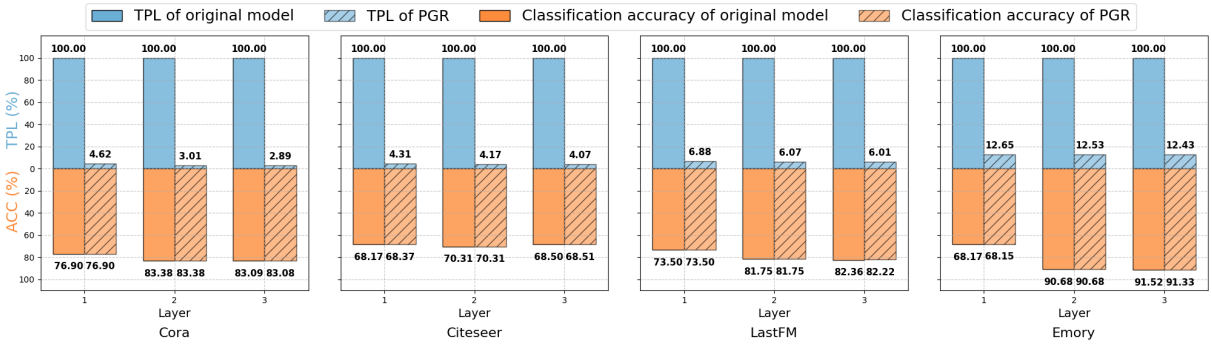


Figure 12: Impact of the number of GNN layers on both topology privacy leakage (TPL) and model accuracy. The x-axis indicates the number of GCN layers. The y-axis denotes both TPL (in blue color) and model accuracy (in orange color).

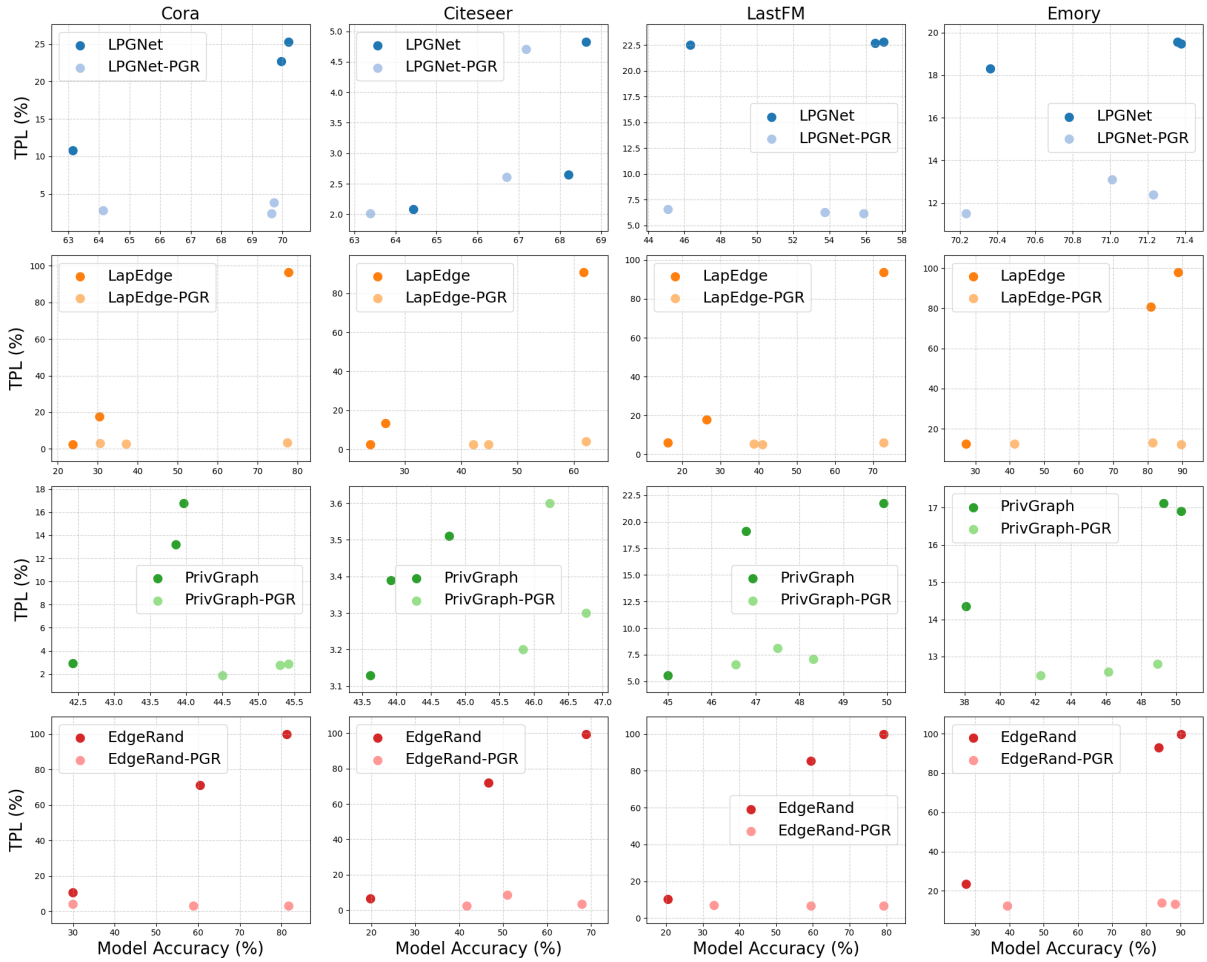


Figure 13: The trade-off between TPL and model accuracy of DP-PGR adapted from four edge-DP approaches (LPGNet [19], PrivGraph [39], LapEdge [37], and EdgeRand [37]). The x-axis and y-axis indicate model accuracy (%) and topology privacy leakage (TPL), respectively. We use the privacy budget $\epsilon \in \{1, 5, 9\}$ for all the algorithms and datasets.